

Peretto

marcos
Luciano

PDI — ATIVIDADE 01 · REGISTRO DE ENTREGA

Sistemas Distribuídos com **Mensageria** (fila, tópico, DLQ)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company — Automação & Infraestrutura

Módulo: 05 · Sistemas Distribuídos, Mensageria e Eventos

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

filas / RabbitMQ / Kafka

1. Contexto

A FV precisa que o time de **Vendas** registre um negócio e, em segundos, o time de **Financeiro**, o de **Marketing** e o de **Operação** reajam — sem que um sistema espere o outro travar. Hoje tudo é feito por chamada síncrona: o Vendas chama o Financeiro, que chama o Marketing... se um cai, a cadeia inteira falha. É o chamado **acoplamento forte**.

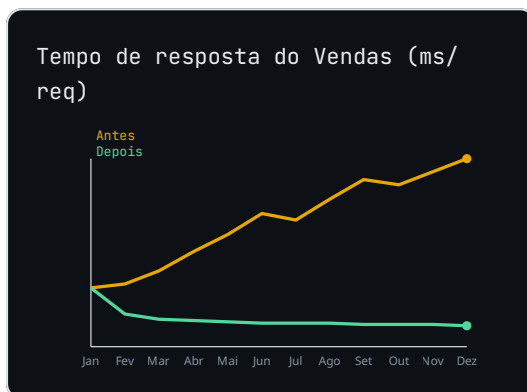
ANALOGIA DO RESTAURANTE

Imagine um restaurante onde o garçom só libera o pedido se ele mesmo levar o prato à cozinha, encher o copo, cobrar no caixa e lavar a louça antes de atender a próxima mesa. Ninguém come. A **mensageria** é o comanda na cozinha: o garçom anota o pedido

(evento) e segue. A cozinha, o caixa e a lavanderia lêem a comanda no próprio ritmo. Cada setor trabalha independente.

2. Diagnóstico

Com chamadas síncronas, uma lentidão no Financeiro trava o Vendas. Picos de campanha geram **fila invisível** de requisições perdidas. Não há replay: se o Marketing estava fora do ar, o evento sumiu. O custo é operacional (timeout em cascata) e de oportunidade (lead esquecido).



Raiz do problema: o Vendas é acoplado ao Financeiro/Marketing via chamada direta. Não existe **desacoplamento** nem **buffer** para absorver picos — o sistema não escala porque cada parte depende do tempo de resposta da outra.

3. Solução

Inserir um **broker de mensageria** (RabbitMQ para filas de trabalho, Kafka para eventos de fluxo) entre os sistemas. O Vendas publica um evento `venda.criada` e segue em ~2ms. Cada time assina o que precisa. Há dois padrões principais: **fila (queue)**, onde 1 mensagem é consumida por 1 worker (tarefa), e **publish/subscribe**, onde 1 mensagem é entregue a N assinantes (evento difundido).

4. Como funciona (pipeline)

Pipeline de mensageria venda.criada

FLUXO EM EXECUÇÃO

1

Producer `vendas.publish()`

Vendas publica `venda.criada` no broker e retorna imediatamente (fire-and-forget).

`ok`

2

Broker `exchange/partition`

Roteia por tópico: fila de cobrança (1 worker) e fan-out de eventos (N assinantes).

`ok`

3

Consumer Financeiro `cobranca.consume()`

Consome da fila, gera boleto, faz `ack`.

`warn`

4

Consumer Marketing + Operação `*.on_event()`

Assinam o mesmo tópico em pub/sub: dispara campanha e abre ordem.

`ok`

5

Dead Letter `dlq.save()`

Falha de parse vira para DLQ para reprocessamento sem perder evento.

`audit`

Pico de campanha → broker absorve o volume; cada consumer processa no próprio ritmo (back-pressure natural).

5. Antes vs Depois

CENÁRIO

ANTES (SÍNCRONO)

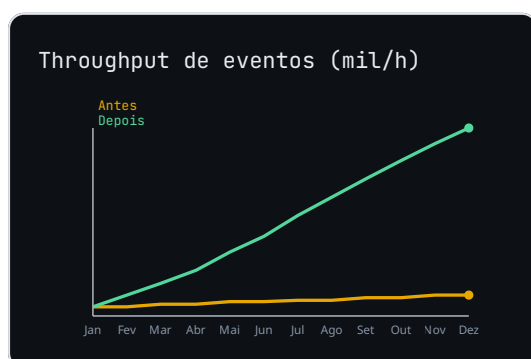
DEPOIS (MENSAGERIA)

Falha em 1 sistema	Trava a cadeia	Isolado ao consumer
Pico de campanha	Timeouts / perda	Buffer no broker
Replay de evento	Impossível	Replay por offset (Kafka)
Acoplamento	Forte (Vendas→Fin)	Fraco (evento)

6. Entregas

- **Diagrama** `topologia-mensageria.png`: fila vs pub/sub com os 3 times.
- **Worker** `cobranca_consumer.py`: consumer com ack e DLQ.
- **Producer** `vendas_publisher.py`: publicação assíncrona via RabbitMQ.
- **Doc** `runbook-mensageria.md`: como reprocessar a DLQ e subir um consumer.

7. Métricas



Meta: 100% dos eventos de venda processados de forma assíncrona, com zero perda em pico e P95 de ponta-a-ponta < 5s.

8. Status final

NÃO PUBLICADO — Desenvolvido e em homologação. Aguarda revisão antes de produção.